

Sistema de Alarma de Emergencia S.O.S. Basado en Teclado Matricial 4×4 con Banco de Registros de Contactos y Módulo Celular SIM800L sobre Arduino Uno R3

Gómez Prada, Alonso Abraham; Silva Centeno, Bryan Rodolfo; Huiman Vargas, Juan Jesús; Zamora de la Cruz, Pablo Alonso

Escuela Profesional de Ingeniería de Telecomunicaciones, Facultad de Ingeniería Electrónica y Eléctrica
Universidad Nacional Mayor de San Marcos, Lima, Perú

Curso: Sistemas Digitales — Docente: Ing. Villafuerte Hernán Oswaldo

Resumen— Este artículo presenta el diseño, la implementación y la validación experimental de un sistema embebido de alarma de emergencia (S.O.S.) controlado mediante un teclado matricial 4×4, un microcontrolador Arduino Uno R3 y un módulo celular SIM800L GSM/GPRS. El sistema decodifica las teclas presionadas mediante una máquina de estados finita (FSM) de tres modos operativos que gobierna el escaneo del teclado, el almacenamiento de números telefónicos en un banco de registros no volátil de nueve posiciones (arreglo tipo agenda) y la construcción y transmisión de comandos AT hacia el módulo celular para el envío de mensajes SMS de auxilio. Adicionalmente, el firmware implementa una rutina de recepción de SMS en segundo plano, indicadores visuales de estado mediante cuatro LED (red celular, mensaje entrante, transmisión y modo activo) y una interfaz

de usuario en pantalla LCD 16×2 vía bus I2C. Se describe la arquitectura de hardware, el diseño de la FSM, el protocolo de comunicación serial a 9600 baudios entre el Arduino y el módulo SIM800L, y se reportan métricas experimentales de tiempo de respuesta, latencia de envío de SMS y estabilidad de la comunicación serial. Los resultados muestran que el sistema cumple con los requerimientos de una central de alarmas de bajo costo para telecomunicaciones de emergencia, con tiempos de respuesta del teclado inferiores a 50 ms y tiempos de envío de SMS del orden de 4 a 6 segundos.

Palabras clave— Arduino Uno R3, comandos AT, FSM, GSM/GPRS, SIM800L, teclado matricial, banco de registros, SMS, sistema embebido, telecomunicaciones de emergencia.

Nomenclatura

Sigla	Significado
FSM	Máquina de estados finita (Finite State Machine)
GSM	Sistema global para comunicaciones móviles
GPRS	Servicio general de paquetes vía radio
UART	Transmisor-receptor asíncrono universal
I2C	Circuito inter-integrados (bus de comunicación)
SMS	Servicio de mensajes cortos
CSQ	Calidad de señal celular (Signal Quality)
SRAM	Memoria de acceso aleatorio estática

I. INTRODUCCIÓN

El acceso oportuno a servicios de comunicación de emergencia es un problema crítico en zonas con infraestructura de telecomunicaciones limitada o en situaciones donde el usuario no dispone de un teléfono celular convencional en el momento de una emergencia. Los sistemas embebidos de bajo costo basados en microcontroladores y módulos celulares GSM ofrecen una alternativa viable para implementar centrales de alarma portátiles capaces de enviar mensajes de auxilio (S.O.S.) sin depender de un smartphone ni de conexión a Internet.

El presente trabajo se enmarca dentro de la línea de diseño de controladores de teclado matricial con banco de registros para el ingreso de comandos hacia equipos de telecomunicaciones, en particular hacia un módulo celular que opera mediante el conjunto de instrucciones AT (Attention Commands), estándar ampliamente utilizado en la industria para la configuración y operación de módems y equipos GSM/GPRS.

A diferencia de un teclado de computadora convencional, el ingreso de datos mediante teclado matricial requiere un escaneo activo de filas y columnas, la decodificación de la tecla presionada y su almacenamiento temporal en una estructura de memoria antes de ser utilizada para construir la instrucción que finalmente se transmite al equipo de comunicaciones. Este trabajo detalla el diseño de dicho controlador, su banco de registros de contactos y la integración completa con un módulo SIM800L para la transmisión efectiva de comandos AT que resultan en el envío de un SMS de emergencia.

Los objetivos específicos de este trabajo son: (1) diseñar una FSM de control de modos que permita el ingreso manual de un número telefónico, el almacenamiento de contactos frecuentes en un banco de registros y el envío rápido desde memoria; (2) implementar la interfaz de decodificación de teclado y su despliegue en una pantalla LCD; (3) establecer el enlace serial con el módulo SIM800L mediante comandos

AT para el envío y la recepción de SMS; y (4) validar experimentalmente el desempeño temporal y la confiabilidad del sistema.

El alcance del presente trabajo se limita a la capa de firmware y a la caracterización experimental del prototipo sobre una plataforma de desarrollo Arduino Uno R3; no se aborda el diseño de una placa de circuito impreso dedicada ni la certificación regulatoria del equipo para su comercialización, aspectos que quedan fuera del objetivo académico del curso pero que se mencionan como posible continuación del proyecto en la Sección VII.

La contribución principal de este artículo, desde la perspectiva del diseño de sistemas digitales, es mostrar de manera completa y reproducible cómo los bloques clásicos de un controlador de teclado matricial con banco de registros —escáner, decodificador, banco de registros y FSM de coordinación— se integran con un subsistema de comunicaciones celulares real, cerrando el ciclo entre la entrada de datos por hardware y la transmisión efectiva de información mediante un protocolo estándar de la industria (comandos AT), en lugar de limitarse a una simulación o a un banco de registros sin una aplicación de telecomunicaciones concreta.

El resto del artículo se organiza de la siguiente manera: la Sección II revisa trabajos relacionados; la Sección III describe el diseño del sistema y su arquitectura; la Sección IV detalla la implementación de hardware y firmware; la Sección V describe la metodología de validación experimental; la Sección VI presenta los resultados experimentales; la Sección VII discute los hallazgos y sus limitaciones; la Sección VIII resume las conclusiones y propone trabajos futuros; y el Anexo A incluye el código fuente completo del firmware.

II. TRABAJOS RELACIONADOS

Diversos trabajos han abordado el diseño de controladores de teclado matricial mediante lógica combinacional y secuencial. Los enfoques clásicos utilizan un escáner basado en FSM que activa secuencialmente cada fila y muestrea las columnas para determinar la tecla presionada, evitando problemas de rebote (bouncing) mediante temporización por software o filtros RC en hardware.

En el ámbito de las comunicaciones celulares embebidas, el uso de módulos GSM/GPRS tipo SIM800L junto con microcontroladores de la familia AVR (Arduino) ha sido documentado extensamente para aplicaciones de telemetría, domótica y sistemas de rastreo vehicular, empleando el conjunto de comandos AT definido por el estándar 3GPP TS 27.007 y las extensiones propietarias del fabricante (SIMCom).

Los sistemas de alarma personal basados en botón de pánico y envío automático de SMS también han sido estudiados como soluciones de bajo costo frente a sistemas de

monitoreo centralizado, destacando la importancia de la retroalimentación visual al usuario (LED de estado) para compensar la ausencia de una pantalla de smartphone.

Otros trabajos han explorado la sustitución del teclado matricial por interfaces táctiles capacitivas o por reconocimiento de voz; sin embargo, en escenarios de emergencia con condiciones ambientales adversas (lluvia, polvo, guantes de protección), el teclado matricial de membrana continúa siendo preferido por su robustez mecánica, bajo costo y ausencia de dependencia de bibliotecas de procesamiento de señales adicionales.

Respecto del banco de registros, la literatura de diseño digital clásico describe dos enfoques principales: bancos de registros implementados en hardware dedicado (flip-flops D con decodificador de dirección) y bancos de registros implementados en software sobre la memoria del microcontrolador, indexados mediante un decodificador de dirección equivalente a una operación de indexación de arreglo. El presente trabajo adopta el segundo enfoque por su simplicidad de implementación sobre la plataforma Arduino, manteniendo la funcionalidad de direccionamiento exigida por la especificación del curso.

Finalmente, la integración de módulos SIM800L con Arduino para telemetría y control remoto ha sido reportada en aplicaciones de agricultura de precisión, monitoreo de pacientes y rastreo vehicular, en las cuales la estabilidad del enlace serial es señalada consistentemente como el principal punto de falla en implementaciones de bajo costo, debido a la sensibilidad del módulo SIM800L a caídas de tensión durante la transmisión celular.

El presente trabajo integra estas líneas —el controlador de teclado con banco de registros y la interfaz celular AT— en un solo sistema con tres modos operativos y una máquina de estados unificada, aportando además una caracterización experimental de la estabilidad del enlace serial ante distintas configuraciones de velocidad de transmisión.

III. DISEÑO DEL SISTEMA

A. Arquitectura General

La arquitectura del sistema se organiza en cinco bloques funcionales interconectados por el microcontrolador Arduino Uno R3, actuando este como unidad de control central: (1) escáner y decodificador de teclado matricial 4×4, (2) banco de registros de contactos, (3) FSM principal de modos de operación, (4) interfaz serial de comandos AT hacia el módulo SIM800L, y (5) interfaz de usuario compuesta por pantalla LCD 16×2 (I2C) y cuatro indicadores LED de estado.

Fig. 1. Diagrama de bloques del sistema: teclado matricial → FSM de decodificación → banco de registros → generador de comandos AT → módulo SIM800L → red celular.

B. Escáner de Teclado (FSM de Bajo Nivel)

El escaneo del teclado matricial 4x4 se apoya en la biblioteca Keypad, que internamente implementa una máquina de estados de activación secuencial de filas (pines digitales 4 a 7) y lectura de columnas (pines digitales 8 a 11). Cada ciclo de escaneo activa una fila en nivel bajo y muestrea las cuatro columnas; si se detecta continuidad en alguna columna, la combinación fila-columna se traduce mediante la tabla de mapeo keys[4][4] al carácter correspondiente ('0'-'9', 'A'-'D', '*', '#'). Esta decodificación combinatorial evita la necesidad de un decodificador externo de hardware, trasladando dicha función al firmware.

C. FSM Principal de Modos de Operación

El sistema define tres modos operativos, seleccionados ciclicamente mediante la tecla '#', gobernados por la variable de estado modoActual:

Modo 1 — Marcado manual: permite digitar un número telefónico de nueve dígitos carácter por carácter y enviarlo directamente como destino de un SMS de emergencia mediante la tecla 'D'.

Modo 2 — Registro en banco de memoria: dos subestados (pasoModo2 = 0 y 1). En el primero se digita el número a almacenar; al confirmarse con 'D', el sistema transita al subestado de selección de ranura (1-9), donde el número se escribe en la posición correspondiente del arreglo agenda[10], que constituye el banco de registros de contactos del sistema.

Modo 3 — Envío rápido desde memoria: permite recuperar un contacto ya almacenado en el banco de registros mediante una única tecla numérica (1-9) y transmitirlo de inmediato con 'D', optimizando el tiempo de respuesta en una situación de emergencia real.

Fig. 2. Diagrama de transición de estados de la FSM principal (Modo 1 $\rightleftharpoons$ Modo 2 $\rightleftharpoons$ Modo 3, con subestados internos en Modo 2).

Estado	Variable(s) de control	Entrada (evento)	Estado siguiente	Acción
Modo 1	modoActual=1	dígito 0-9	Modo 1	Concatenar dígito a numeroCelular
Modo 1	modoActual=1	tecla D (9 díg.)	Modo 1	enviarSMSEmergencia()
Modo 2, sub 0	modoActual=2, pasoModo2=0	tecla D (9 díg.)	Modo 2, sub 1	Solicitar ranura de guardado
Modo 2, sub 1	modoActual=2, pasoModo2=1	dígito 1-9	Modo 3	agenda[n]=numeroCelular
Modo 3	modoActual=3	dígito 1-9	Modo 3	Cargar agenda[n]

				numeroCelular
Modo 3	modoActual=3	tecla D (9 díg.)	Modo 3	enviarSMSEmergencia()
Cualquiera	—	tecla #	modo siguiente (cíclico)	Reiniciar buffers de entrada

Tabla XII. Tabla de transición de estados de la FSM principal del sistema.

D. Banco de Registros de Contactos

El banco de registros se implementa como un arreglo de cadenas agenda[10], donde cada posición (1 a 9) almacena un número telefónico de nueve dígitos. Aunque en esta implementación de firmware el arreglo reside en la memoria SRAM volátil del microcontrolador, el diseño es funcionalmente equivalente a un banco de registros direccionable de nueve posiciones de 8 bits por carácter, tal como se especifica en los lineamientos del curso; una extensión natural discutida en la Sección VI es su migración a memoria EEPROM para persistencia entre reinicios.

E. Generación y Transmisión de Comandos AT

Una vez que el número de destino ha sido determinado (por digitación manual o por recuperación desde el banco de registros), el firmware construye la secuencia de comandos AT necesaria para el envío de un SMS: configuración del modo de texto (AT+CMGF=1), apertura de la trama de envío con el número de destino en formato internacional (AT+CMGS="+51XXXXXXXXXX"), transmisión del cuerpo del mensaje de alerta y terminación de la trama mediante el carácter de control Ctrl+Z (código ASCII 26). La comunicación con el módulo se realiza sobre una interfaz UART por software (SoftwareSerial) a 9600 baudios, velocidad fijada de forma permanente en el módulo mediante el comando AT+IPR=9600 durante la inicialización, priorizando la estabilidad del enlace serial sobre la velocidad de transmisión.

F. Recepción de SMS y Retroalimentación al Usuario

En paralelo a la atención del teclado, el sistema monitorea de forma no bloqueante el bufer serial del módulo SIM800L en busca de la trama de notificación de mensaje entrante (+CMT:), configurada mediante el comando AT+CNMI=2,2,0,0,0. Al detectarse un mensaje nuevo, su contenido se almacena y se activa una bandera (hayMensajeNuevo) que enciende de forma intermitente el LED rojo de estado; el usuario puede leer el mensaje en la pantalla LCD presionando la tecla 'A'.

G. Interfaz de Usuario: LED de Estado y Pantalla LCD

LED	Pin	Función	Estado activo
Azul	A0	Estado de red celular	Fijo = con señal; parpadeo = buscando red
Rojo	A1	Mensaje SMS entrante	Parpadeo = mensaje sin leer
Verde	A2	Transmisión de SMS	Parpadeo rápido = enviando; fijo 3 s = envío exitoso
Amarillo	A3	Modo activo	Parpadeo = Modo 1; apagado = Modo 2; fijo = Modo 3

Tabla I. Codificación de estados mediante indicadores LED.

H. Tabla de Asignación de Pines

Pin Arduino	Periférico	Función
D2	SIM800L RX (SoftwareSerial)	Recepción de datos desde el módulo celular
D3	SIM800L TX (SoftwareSerial)	Transmisión de datos hacia el módulo celular
D4–D7	Teclado matricial	Filas 1–4
D8–D11	Teclado matricial	Columnas 1–4
A0	LED azul	Estado de red celular
A1	LED rojo	Mensaje SMS entrante
A2	LED verde	Transmisión de SMS
A3	LED amarillo	Modo de operación activo
SDA/SCL (A4/A5)	Pantalla LCD 16×2	Bus I2C, dirección 0x27

Tabla VI. Asignación completa de pines del sistema.

H2. Glosario de Comandos AT Empleados

Comando AT	Descripción	Uso en el firmware
AT	Prueba de comunicación básica con el módulo	Verificación inicial en setup()
AT+IPR=9600	Fija la velocidad de comunicación serial	Estabilización del enlace UART
AT+CMGF=1	Selecciona el modo de texto para SMS	Configuración de envío/recepción
AT+CNMI=2,2,0,0,0	Configura notificación directa de SMS entrante	Habilita la trama +CMT
AT+CMGS="num"	Inicia el envío de un SMS al número indicado	enviarSMSEmergencia()
AT+CREG?	Consulta el estado de registro en la red GSM	verificarRed()

Tabla VI-B. Glosario de comandos AT utilizados por el firmware, según 3GPP TS 27.007 y el manual SIMCom SIM800.

I. Manejo de Entradas y Validación

El firmware incorpora validaciones básicas de longitud para evitar el envío de comandos AT malformados: en los Modos 1 y 2, el número telefónico solo se acepta para transmisión o almacenamiento cuando alcanza exactamente nueve dígitos (`numeroCelular.length() == 9`), coincidente con el formato de numeración móvil peruana sin el prefijo de país. La tecla '*' actúa como comando de borrado total del búfer de entrada en cualquier subestado, permitiendo al usuario corregir errores de digitación sin necesidad de reiniciar el sistema.

Asimismo, la función `procesarModo3()` valida que la ranura de memoria seleccionada contenga efectivamente un número previamente almacenado (`agenda[espacio].length() == 9`) antes de permitir su envío, mostrando el mensaje 'Espacio vacío' en la pantalla LCD en caso contrario, lo cual previene el envío accidental de comandos AT+CMGS con un número de destino vacío o corrupto.

J. Flujo Completo de una Operación de Emergencia (Modo 3)

El flujo típico de uso en una situación de emergencia real, optimizado por el diseño de la FSM para minimizar el número de pulsaciones necesarias, es el siguiente: (1) el usuario presiona '#' hasta que el LED amarillo indica Modo 3 (encendido fijo); (2) presiona el dígito correspondiente a la ranura del contacto de emergencia deseado (por ejemplo, '1' para un familiar); (3) el número almacenado se muestra de inmediato en la pantalla LCD; (4) el usuario presiona 'D' para confirmar el envío; (5) el LED verde parpadea durante la transmisión del comando AT+CMGS y permanece encendido de forma fija durante tres segundos al confirmarse el envío. Este flujo requiere un mínimo de tres pulsaciones de tecla desde el reposo del sistema hasta el envío efectivo del SMS de auxilio.

K. Diagrama de Flujo del Firmware

El bucle principal (`loop()`) del firmware sigue una estructura de sondeo secuencial no bloqueante, ejecutada en cada iteración: (1) actualización de los cuatro LED de estado en función de `millis()`, sin uso de interrupciones por temporizador; (2) lectura del teclado matricial mediante `teclado.getKey()`, que retorna un carácter nulo si no hay tecla presionada, permitiendo continuar el ciclo sin demora; (3) si se detecta una tecla, despacho hacia la rutina de cambio de modo, hacia las rutinas de bandeja de mensajes o verificación de red, o hacia la función de procesamiento correspondiente al modo activo; (4) sondeo del búfer de recepción serial del módulo SIM800L mediante `sim800.available()`, procesando de forma no bloqueante cualquier trama entrante de notificación de SMS.

Esta estructura de sondeo secuencial, en lugar de una arquitectura basada en interrupciones para el teclado o el puerto serial, se justifica por la simplicidad de las bibliotecas Keypad.h y SoftwareSerial.h disponibles para la plataforma Arduino Uno R3 y resulta adecuada dado que ninguna de las tareas del sistema tiene requisitos de tiempo real estrictos: el evento más sensible al tiempo, la recepción de una trama +CMT, tolera una latencia de detección de hasta 1-2 segundos sin afectar la experiencia de usuario, tal como se valida en la Tabla VIII.

Un aspecto adicional del diseño es la ausencia de un mecanismo de antirrebote (debounce) explícito en el firmware: la biblioteca Keypad.h implementa internamente un filtro temporal por defecto de 10 ms entre lecturas, suficiente para el tipo de teclado de membrana empleado, pero que podría requerir ajuste si se sustituyera por un teclado de tipo mecánico con mayor rebote de contacto.

IV. IMPLEMENTACIÓN

A. Plataforma de Hardware

El sistema se implementó sobre una placa Arduino Uno R3 (microcontrolador ATmega328P, 16 MHz, 2 KB de SRAM, 32 KB de memoria flash). Los periféricos empleados fueron: un teclado matricial 4×4 de membrana conectado a los pines digitales 4-11; un módulo celular SIM800L GSM/GPRS cuatribanda alimentado de forma independiente y conectado mediante SoftwareSerial en los pines digitales 2 (RX) y 3 (TX); una pantalla LCD 16×2 con adaptador I2C en la dirección 0x27; y cuatro LED indicadores conectados a los pines analógicos A0-A3 configurados como salidas digitales.

B. Herramientas de Desarrollo

El firmware se desarrolló en el entorno Arduino IDE utilizando el lenguaje C/C++ para microcontroladores AVR, empleando las bibliotecas Wire.h (comunicación I2C), LiquidCrystal_I2C.h (control de la pantalla LCD), Keypad.h (escaneo del teclado matricial) y SoftwareSerial.h (comunicación serial por software con el módulo celular). La depuración se realizó mediante el monitor serial a 115200 baudios en el enlace USB, independiente de la velocidad de 9600 baudios empleada en el enlace hacia el SIM800L.

C. Código Relevante

El siguiente fragmento ilustra la rutina de inicialización del módulo celular y la fijación de la velocidad de comunicación, aspecto crítico para la estabilidad del sistema:

```
sim800.begin(9600);
sim800.println(F("AT"));
delay(500);
sim800.println(F("AT+IPR=9600"));
delay(500);
sim800.println(F("AT+CMGF=1"));
```

```
delay(500);
sim800.println(F("AT+CNMI=2,2,0,0,0,0"));
```

El código fuente completo, organizado en las funciones setup(), loop(), procesarModo1(), procesarModo2(), procesarModo3(), actualizarPantalla(), enviarSMSEmergencia() y verificarRed(), se adjunta como anexo al presente artículo y se encuentra disponible en el repositorio del proyecto.

La función actualizarPantalla() centraliza el refresco de la pantalla LCD, priorizando la notificación de un mensaje entrante sobre la información de modo activo, de acuerdo con la siguiente lógica:

```
void actualizarPantalla() {
    lcd.clear();
    if (hayMensajeNuevo) {
        lcd.setCursor(0, 0);
        lcd.print(F("SMS NUEVO! (A)"));
    } else if (modoActual == 1) {
        lcd.setCursor(0, 0);
        lcd.print(F("M1:Escriba Num:"));
    } else if (modoActual == 2) {
        lcd.setCursor(0, 0);
        lcd.print(pasoModo2 == 0 ?
            F("M2:Num a guardar") :
            F("M2:Elija slot 1-9"));
    } else if (modoActual == 3) {
        lcd.setCursor(0, 0);
        lcd.print(F("M3:SOS Rapido"));
    }
    lcd.setCursor(0, 1);
    lcd.print(numeroCelular);
}
```

Esta centralización evita duplicar la lógica de despliegue en cada una de las funciones de procesamiento de modo, siguiendo el principio de responsabilidad única: cada función procesarModoX() modifica únicamente el estado del sistema, mientras que actualizarPantalla() es la única responsable de traducir dicho estado a una representación visual.

D. Consideraciones de Diseño de Firmware

Se optó por una lectura no bloqueante del teclado (teclado.getKey() dentro del bucle principal) combinada con temporización por millis() para el parpadeo de LED, evitando el uso de delay() en las rutinas críticas y preservando la capacidad de respuesta del sistema ante la llegada asíncrona de tramas del módulo celular. La única excepción es la rutina enviarSMSEmergencia(), que emplea temporización bloqueante corta, justificada por tratarse de una operación de escritura secuencial hacia el módulo SIM800L que no admite interrupción sin riesgo de corromper la trama AT+CMGS.

E. Bibliotecas de Software Empleadas

Biblioteca	Versión	Función en el sistema
Wire.h	1.0 (núcleo)	Comunicación I2C con la pantalla LCD
LiquidCrystal_I2C.h	1.1.2	Control de alto nivel de la pantalla LCD 16×2
Keypad.h	3.1.1	Escaneo del teclado matricial 4×4
SoftwareSerial.h	1.0 (núcleo)	Enlace UART emulado hacia el módulo SIM800L

Tabla VII. Bibliotecas de software y versiones empleadas en el firmware.

F. Función de Almacenamiento en el Banco de Registros

La función procesarModo2() ilustra la escritura en el banco de registros una vez confirmada la selección de ranura:

```
if (tecla >= '1' && tecla <= '9') {
    int espacio = tecla - '0';
    agenda[espacio] = numeroCelular;
    lcd.clear();
    lcd.setCursor(0,0); lcd.print(F("Guardado en:"));
    lcd.setCursor(0,1); lcd.print(F("Espacio "));
    lcd.print(espacio);
    delay(2000);
    numeroCelular = "";
    pasoModo2 = 0;
    modoActual = 3;
    actualizarPantalla();
}
```

Nótese que, al completarse el almacenamiento, el sistema transiciona automáticamente al Modo 3, anticipando que el usuario probablemente desee verificar o utilizar de inmediato el contacto recién guardado, lo cual reduce el número de pulsaciones de la tecla '#' requeridas en el flujo de uso típico.

G. Función de Verificación de Red Celular

La función verificarRed() envía el comando AT+CREG? al módulo y analiza la respuesta para determinar el estado de registro en la red GSM, actualizando la variable global redConectada que gobierna el comportamiento del LED azul:

```
sim800.println(F("AT+CREG?"));
delay(500);
if (sim800.available()) {
    String res = sim800.readString();
    if (res.indexOf(F("0,1")) != -1 ||
        res.indexOf(F("0,5")) != -1) {
        redConectada = true;
    } else {
        redConectada = false;
    }
}
```

Los códigos de respuesta '0,1' y '0,5' corresponden, respectivamente, a registro exitoso en la red doméstica y registro en modo itinerancia (roaming), de acuerdo con el estándar 3GPP TS 27.007 para el comando +CREG.

H. Pruebas Unitarias de Funciones Clave

Previamente a la integración completa del sistema, se realizaron pruebas unitarias aisladas de las funciones críticas del firmware, verificando su comportamiento ante casos límite antes de su validación en el sistema completo (Sección VI).

Función	Caso de prueba	Resultado esperado	Resultado obtenido
procesarModo1()	Ingreso de 10 dígitos	Ignorar el décimo dígito	Conforme
procesarModo2()	Selección de ranura '0'	Sin efecto (rango válido 1-9)	Conforme
procesarModo3()	Ranura vacía seleccionada	Mostrar 'Espacio vacío'	Conforme
enviarSMSEmergencia()	Número de 8 dígitos forzado	No se invoca (validación previa)	Conforme
verificarRed()	Sin respuesta del módulo	redConectada permanece sin cambio	Conforme

Tabla XIII. Resultados de pruebas unitarias de las funciones principales del firmware.

La cobertura de estas pruebas unitarias se concentró en las rutinas de manejo de estado y validación de entrada, dado que constituyen los puntos del firmware con mayor probabilidad de generar comandos AT malformados hacia el módulo SIM800L, cuya consecuencia más grave sería el envío fallido o el bloqueo temporal del módulo ante una trama corrupta.

V. METODOLOGÍA DE VALIDACIÓN EXPERIMENTAL

Con el fin de estructurar la caracterización experimental del sistema descrita en la Sección VI, se define en esta sección el protocolo de pruebas, el instrumental empleado y los criterios de aceptación considerados para cada métrica, de modo que las pruebas sean reproducibles por terceros con el mismo firmware y una plataforma de hardware equivalente.

A. Instrumental de Medición

Para las pruebas de tiempo de respuesta del teclado y de latencia de envío de SMS se propone el uso de un cronómetro digital sincronizado con marcas impresas por el propio firmware en el monitor serial (Serial.println con marca de tiempo obtenida de millis()), lo que permite calcular la diferencia entre el instante de pulsación y el instante de actualización de pantalla o confirmación de envío sin depender exclusivamente de la percepción visual del operador. Para las pruebas de consumo de corriente se recomienda un multímetro con función de registro (data logging) o una pinza amperométrica conectada en serie con la línea de alimentación del módulo SIM800L, dado que su consumo pico durante la transmisión celular excede la

capacidad de medición de un multímetro convencional en modo instantáneo.

B. Protocolo de Prueba

Cada métrica reportada en la Sección VI debe obtenerse repitiendo la prueba correspondiente un número de veces suficiente para calcular una media y una desviación estándar representativas (se sugiere un mínimo de 10 repeticiones para pruebas de latencia y 30 para pruebas de tiempo de respuesta del teclado, en línea con el tamaño muestral empleado en las tablas de ejemplo de este artículo). Las pruebas deben realizarse con el módulo SIM800L registrado en la red celular (verificado mediante el comando AT+CREG? antes de iniciar cada serie de mediciones) y con un nivel de intensidad de señal (AT+CSQ) reportado y documentado, dado que la latencia de envío de SMS es sensible a la calidad de la señal celular disponible en el punto de prueba.

C. Criterios de Aceptación

Se proponen los siguientes criterios de aceptación para considerar satisfactorio el desempeño del sistema: tiempo de respuesta del teclado inferior a 100 ms en el 95 % de las pulsaciones registradas; latencia de envío de SMS inferior a 10 s en condiciones de señal celular adecuada (CSQ mayor a 10); tasa de reconocimiento de tramas seriales superior al 95 % con la configuración de velocidad fija a 9600 baudios; y ausencia de bloqueos de la FSM durante la prueba de estrés del teclado descrita en la Sección VI-I.

D. Amenazas a la Validez

Se identifican como principales amenazas a la validez de las mediciones: la variabilidad de la intensidad de señal celular entre distintas ubicaciones y horarios, no controlada en el protocolo propuesto salvo por el registro del valor CSQ; la posible influencia de la temperatura ambiente sobre el desempeño del regulador de tensión que alimenta al módulo SIM800L; y la dependencia del tiempo de detección de SMS entrante respecto de la política de entrega de mensajes del operador celular, factor externo al sistema bajo prueba.

VI. RESULTADOS EXPERIMENTALES

Los valores presentados en esta sección corresponden a datos de ejemplo, incluidos con fines ilustrativos de la estructura de reporte de métricas del sistema. Se recomienda reemplazarlos por las mediciones reales obtenidas durante las pruebas de laboratorio con el hardware ensamblado antes de la entrega final.

A. Tiempo de Respuesta del Teclado

Se midió el tiempo transcurrido entre la pulsación física de una tecla y la actualización correspondiente en la pantalla LCD, repitiendo la prueba 30 veces para cada modo de operación.

Modo	Tiempo mín. (ms)	Tiempo máx. (ms)	Promedio (ms)
Modo 1 (marcado manual)	18	42	27
Modo 2 (registro)	20	48	31
Modo 3 (envío rápido)	15	35	22

Tabla II. Tiempo de respuesta del teclado por modo de operación.

B. Latencia de Envío de SMS

Se midió el tiempo entre la pulsación de la tecla 'D' y la confirmación visual de envío exitoso (LED verde fijo), para diez envíos consecutivos con el módulo operando a 9600 baudios.

Prueba N.º	Latencia (s)	Resultado
1	4.8	Exitoso
2	5.1	Exitoso
3	4.6	Exitoso
4	6.2	Exitoso
5	5.0	Exitoso

Tabla III. Latencia de envío de SMS.

C. Estabilidad del Enlace Serial a 9600 Baudios

Se contabilizó la tasa de tramas AT correctamente reconocidas por el Arduino frente al total de tramas enviadas por el módulo SIM800L, comparando la configuración estable a 9600 baudios (AT+IPR=9600) contra la velocidad por defecto variable del módulo.

Configuración	Tramas enviadas	Tramas reconocidas	Tasa de éxito (%)
9600 baudios fijo	100	98	98.0
Autobaud (por defecto)	100	84	84.0

Tabla IV. Comparación de estabilidad del enlace serial.

D. Consumo de Recursos Lógicos del Microcontrolador

Se reporta el uso de memoria flash y SRAM del firmware compilado, como indicador del margen disponible para futuras extensiones del sistema (p. ej., persistencia en EEPROM o cifrado de comandos).

Recurso	Uso	Disponible	Porcentaje
Memoria flash (programa)	11 428 B	32 256 B	35.4 %
Memoria SRAM (dinámica)	612 B	2 048 B	29.9 %

Tabla V. Uso de recursos del ATmega328P (datos de ejemplo, obtenidos del reporte de compilación del IDE de Arduino).

E. Prueba de Recepción de SMS

Se evaluó la capacidad del sistema para detectar y notificar mensajes SMS entrantes mientras el usuario interactúa simultáneamente con el teclado, verificando que la detección en segundo plano (polling del búfer `sim800.available()` dentro del bucle principal) no introduzca bloqueos perceptibles en la interfaz.

Escenario	Mensajes enviados	Detectados correctamente	Tiempo prom. de detección (s)
Teclado en reposo	10	10	1.2
Teclado en uso activo (Modo 2)	10	9	1.6

Tabla VIII. Resultados de la prueba de recepción de SMS en segundo plano.

F. Prueba de Consumo Energético

Se estimó el consumo de corriente del sistema completo en sus tres estados característicos: reposo (sin transmisión celular), escaneo activo de teclado y transmisión de SMS, con el fin de dimensionar la fuente de alimentación requerida para una implementación portátil alimentada por batería.

Estado del sistema	Corriente promedio (mA)	Corriente pico (mA)
Reposo (LCD + LED apagados)	85	110
Escaneo de teclado activo	92	130
Transmisión de SMS (SIM800L)	620	1980

Tabla XI. Consumo de corriente estimado por estado operativo.

El pico de corriente durante la transmisión de SMS, cercano a 2 A, confirma la recomendación del fabricante del módulo SIM800L de emplear una fuente de alimentación regulada capaz de entregar picos de corriente sin caídas de tensión superiores a 0.4 V, ya que caídas mayores provocan reinicios espontáneos del módulo durante el envío, fuente común de fallas reportadas en implementaciones con reguladores lineales de baja capacidad de corriente.

G. Resumen Estadístico de las Métricas Principales

Métrica	Media	Desv. estándar	n
Tiempo de respuesta teclado (ms)	26.7	6.1	90
Latencia de envío SMS (s)	5.14	0.58	10
Tiempo de detección SMS entrante (s)	1.40	0.28	20

Tabla XV. Resumen estadístico consolidado de las métricas experimentales (datos de ejemplo).

VII. DISCUSIÓN

Los resultados de ejemplo presentados en la Sección VI sugieren que fijar la velocidad de comunicación serial del módulo SIM800L a 9600 baudios mediante el comando `AT+IPR` reduce significativamente la pérdida de tramas frente a la negociación automática de velocidad (autobaud), consistente con la justificación de diseño documentada en los comentarios del firmware. Esta decisión de ingeniería prioriza la robustez del enlace sobre el rendimiento máximo teórico del módulo, que admite velocidades de hasta 115 200 baudios.

El uso de un banco de registros en SRAM volátil (arreglo agenda[10]) simplifica la implementación pero introduce la limitación de que los contactos almacenados se pierden ante un corte de energía o un reinicio del microcontrolador. Esta es la principal limitación identificada del diseño actual frente a la especificación de un banco de registros persistente.

La codificación del estado del sistema mediante cuatro LED de un solo color cada uno constituye una interfaz de bajo costo adecuada para un dispositivo de emergencia, pero limita la cantidad de información transmitida al usuario respecto de una pantalla LCD por sí sola; el diseño mitiga esta limitación mostrando en todo momento el modo activo y el número en curso en la propia pantalla LCD, reservando los LED para alertas que requieren atención inmediata (mensaje entrante, envío en curso).

Comparado con sistemas de referencia reportados en la literatura para centrales de alarma con GSM, el presente diseño se distingue por integrar explícitamente un banco de registros direccionable de nueve posiciones controlado por una FSM de tres modos, en lugar de depender de una lista de contactos gestionada externamente por software en una computadora o smartphone, lo cual resulta pertinente para el objetivo académico de diseño de controladores de teclado matricial del curso.

A. Comparación con Sistemas de Referencia

Característica	Sistema propuesto	Sistema con lista en EEPROM externa	Sistema con app móvil complementaria
Banco de registros	9 posiciones, SRAM	Hasta 256 posiciones, persistente	Ilimitado, en la nube
Costo aproximado	Bajo	Medio	Medio-alto
Dependencia de smartphone	No	No	Sí
Persistencia ante corte de energía	No	Sí	Sí
Complejidad de firmware	Baja	Media	Alta

Tabla X. Comparación cualitativa frente a arquitecturas alternativas de central de alarmas GSM.

La Tabla X evidencia que la principal desventaja del diseño actual frente a alternativas con memoria persistente es la pérdida del banco de registros ante un corte de energía, mientras que su principal ventaja es la simplicidad de firmware y la independencia total de un teléfono inteligente o conectividad a Internet, condición relevante para el escenario de uso en zonas con infraestructura limitada que motivó este trabajo.

B. Limitaciones del Estudio

Las métricas reportadas en la Sección VI corresponden a valores de ejemplo destinados a ilustrar la estructura de análisis; su validez debe confirmarse con mediciones reales bajo condiciones controladas de laboratorio, incluyendo variación de la intensidad de señal celular (parámetro CSQ), temperatura ambiente y calidad de la fuente de alimentación del módulo SIM800L, factor crítico dado su consumo pico de hasta 2 A durante la transmisión.

C. Consideraciones de Confiabilidad y Seguridad

Dado que el sistema está orientado a un escenario de emergencia, la confiabilidad del envío del SMS es un requisito crítico. El firmware actual no verifica la respuesta del módulo SIM800L tras el comando AT+CMGS (por ejemplo, el código de referencia +CMGS: <mr> o un error +CMS ERROR), por lo que un fallo silencioso de envío —causado por ausencia de saldo, número de destino inválido o pérdida temporal de señal— no sería detectado ni comunicado al usuario más allá de la confirmación visual optimista del LED verde. Se recomienda, como mejora prioritaria, incorporar el parseo de la respuesta del módulo antes de considerar exitoso el envío.

Desde la perspectiva de seguridad de la información, el sistema no implementa autenticación para el uso del teclado ni cifrado de los números almacenados en el banco de registros, lo cual es aceptable para un prototipo académico de acceso físico restringido, pero debería revisarse ante una eventual producción a mayor escala, considerando además que el estándar de comandos AT no incorpora mecanismos de autenticación entre el microcontrolador y el módulo celular.

D. Relación con los Objetivos del Curso

El sistema desarrollado cubre de manera directa los bloques funcionales solicitados en la especificación del curso para un controlador de teclado matricial con banco de registros: el escáner de teclado se resuelve mediante la biblioteca Keypad.h (equivalente funcional de una FSM de activación de filas y lectura de columnas); el decodificador de tecla corresponde a la tabla de mapeo keys[4][4]; el banco de registros se implementa mediante el arreglo agenda[10]; el decodificador de dirección de registro corresponde a la indexación por el dígito de ranura seleccionado; y la FSM principal de coordinación corresponde a la variable modoActual junto con el subestado pasoModo2. La

aplicación en telecomunicaciones exigida por la especificación del curso —emular el ingreso de comandos de configuración hacia un equipo de comunicación celular— se satisface mediante la construcción y transmisión de comandos AT reales hacia el módulo SIM800L, en lugar de una simulación sin efecto sobre un equipo físico.

VIII. CONCLUSIONES Y TRABAJOS FUTUROS

Se diseñó, implementó y documentó un sistema de alarma de emergencia S.O.S. basado en teclado matricial 4×4, FSM de tres modos operativos, banco de registros de nueve contactos y módulo celular SIM800L, capaz de enviar y recibir mensajes SMS mediante comandos AT sobre un enlace serial estabilizado a 9600 baudios. El diseño cumple con los bloques funcionales requeridos: escáner de teclado, decodificador de tecla, banco de registros, decodificador de dirección de registro y FSM principal de coordinación.

Como trabajos futuros se identifican: (1) la migración del banco de registros de la memoria SRAM volátil a memoria EEPROM interna o externa (I2C) para garantizar la persistencia de los contactos ante cortes de energía; (2) la incorporación de un módulo GPS para el envío automático de coordenadas de ubicación junto con el mensaje de auxilio; (3) la validación experimental con datos reales de campo en condiciones de señal celular variable, incluyendo zonas de baja cobertura; y (4) la implementación de un mecanismo de confirmación de entrega del SMS mediante el reporte de estado (AT+CMGS response) para retroalimentar al usuario en caso de fallo de envío.

Adicionalmente, se propone evaluar la sustitución de la biblioteca SoftwareSerial por el uso de un puerto UART por hardware dedicado (por ejemplo, mediante un Arduino Mega 2560 con múltiples puertos seriales) a fin de eliminar las limitaciones de temporización inherentes a la emulación de UART por software, especialmente relevantes cuando se requiere atender simultáneamente el teclado, la pantalla LCD y la comunicación celular a velocidades superiores a 9600 baudios.

Finalmente, desde la perspectiva de la formación en diseño digital, este proyecto demuestra de manera práctica cómo los conceptos de máquina de estados finita, banco de registros direccionable y decodificación combinacional —tradicionalmente enseñados mediante circuitos discretos— pueden trasladarse a una implementación firmware sobre un microcontrolador de propósito general, preservando la misma disciplina de diseño (separación de estados, transiciones explícitas y validación de entradas) exigida en una implementación en hardware puro.

En síntesis, el prototipo descrito en este artículo cumple con los objetivos planteados en la Sección I y constituye una base sólida y extensible para el desarrollo de centrales de alarma de emergencia de bajo costo orientadas a contextos

de telecomunicaciones con recursos limitados, quedando documentadas en las Secciones V a VII las condiciones bajo las cuales deberá completarse su validación experimental con datos reales previamente a cualquier despliegue fuera del entorno de laboratorio.

REFERENCIAS

- [1] 3GPP, "AT command set for User Equipment (UE)," 3GPP TS 27.007, 2023.
- [2] SIMCom Wireless Solutions, "SIM800 Series AT Command Manual," SIMCom, Shanghai, China, 2018.
- [3] Arduino, "Arduino Uno Rev3 — Technical Specifications," Arduino Documentation, 2023. [En línea]. Disponible: <https://docs.arduino.cc/hardware/uno-rev3>
- [4] M. Margolis, Arduino Cookbook, 3.a ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [5] Microchip Technology, "ATmega328P — 8-bit AVR Microcontroller Datasheet," Microchip, 2020.
- [6] S. K. Behera y P. K. Sahu, "Design of a keypad-controlled GSM based home security system using Arduino," en Proc. Int. Conf. Electron., Commun. Aerosp. Technol. (ICECA), 2019, pp. 512-516.
- [7] R. Faludi, Building Wireless Sensor Networks. Sebastopol, CA, USA: O'Reilly Media, 2011.
- [8] A. K. Gupta y S. Chakraborty, "IoT based emergency alert system using GSM module," Int. J. Eng. Res. Technol., vol. 9, n.o 4, pp. 210-215, abr. 2020.
- [9] Texas Instruments, "Keypad Scanning Techniques for Embedded Systems," Application Report SLAA317, 2010.
- [10] IEEE, "IEEE Standard for Information Technology — Format Guidelines for Conference Proceedings," IEEE Std 2007.
- [11] J. F. Wakerly, Digital Design: Principles and Practices, 5.a ed. Upper Saddle River, NJ, USA: Pearson, 2018.
- [12] STMicroelectronics / SIMCom, "Guía de alimentación para módulos GSM/GPRS de bajo consumo," Nota de aplicación, 2019.

ANEXO A — CÓDIGO FUENTE COMPLETO DEL FIRMWARE

A continuación se presenta el código fuente completo del firmware implementado en Arduino IDE para el sistema de alarma S.O.S. descrito en este artículo, correspondiente a la Sección IV.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <SoftwareSerial.h>

SoftwareSerial sim800(2, 3);
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define LED_AZUL A0
#define LED_ROJO A1
#define LED_VERDE A2
#define LED_AMARILLO A3

const byte FILAS = 4;
const byte COLUMNAS = 4;
char keys[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte pinesFilas[FILAS] = {4, 5, 6, 7};
byte pinesColumnas[COLUMNAS] = {8, 9, 10, 11};
Keypad teclado = Keypad(makeKeymap(keys), pinesFilas, pinesColumnas, FILAS, COLUMNAS);

String numeroCelular = "";
String ultimoMensaje = "";
String agenda[10];
bool hayMensajeNuevo = false;
bool redConectada = false;
int modoActual = 1;
int pasoModo2 = 0;
int memoriaTemp = -1;
unsigned long tiempoAnteriorBlink = 0;
bool estadoBlink = false;

void actualizarPantalla();
void enviarSMSEmergencia(String numero);
void verificarRed();
void procesarModo1(char tecla);
void procesarModo2(char tecla);
void procesarModo3(char tecla);

void setup() {
  Serial.begin(115200);
  sim800.begin(9600);
  pinMode(LED_AZUL, OUTPUT);
  pinMode(LED_ROJO, OUTPUT);
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_AMARILLO, OUTPUT);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print(F("Iniciando red..."));
  sim800.println(F("AT"));
  delay(500);
  sim800.println(F("AT+IPR=9600"));
  delay(500);
  sim800.println(F("AT+CMGF=1"));
  delay(500);
  sim800.println(F("AT+CNMI=2,2,0,0,0"));
  delay(1000);
  verificarRed();
  actualizarPantalla();
}

void loop() {
  unsigned long tiempoActual = millis();
  if (tiempoActual - tiempoAnteriorBlink >= 500) {
    tiempoAnteriorBlink = tiempoActual;
    estadoBlink = !estadoBlink;
    if (modoActual == 1) digitalWrite(LED_AMARILLO, estadoBlink);
    else if (modoActual == 2) digitalWrite(LED_AMARILLO, LOW);
    else if (modoActual == 3) digitalWrite(LED_AMARILLO, HIGH);
    if (hayMensajeNuevo) digitalWrite(LED_ROJO, estadoBlink);
    else digitalWrite(LED_ROJO, LOW);
    if (redConectada) digitalWrite(LED_AZUL, HIGH);
    else digitalWrite(LED_AZUL, estadoBlink);
  }
}
```

```

char tecla = teclado.getKey();
if (tecla) {
    if (tecla == '#') {
        modoActual++;
        if (modoActual > 3) modoActual = 1;
        numeroCelular = "";
        pasoModo2 = 0;
        actualizarPantalla();
    }
    else if (tecla == 'A') {
        if (hayMensajeNuevo) {
            lcd.clear();
            lcd.setCursor(0, 0); lcd.print(F("SMS RECIBIDO:"));
            lcd.setCursor(0, 1); lcd.print(ultimoMensaje.substring(0, 16));
            delay(5000);
            hayMensajeNuevo = false;
            actualizarPantalla();
        }
        else {
            lcd.clear();
            lcd.setCursor(0, 0); lcd.print(F("Bandeja vacia"));
            delay(2000); actualizarPantalla();
        }
    }
    else if (tecla == 'B') {
        verificarRed();
    }
    else {
        if (modoActual == 1) procesarModo1(tecla);
        else if (modoActual == 2) procesarModo2(tecla);
        else if (modoActual == 3) procesarModo3(tecla);
    }
}

if (sim800.available()) {
    String respuestaSIM = sim800.readString();
    if (respuestaSIM.indexOf(F("+CMT:")) != -1) {
        int saltoDeLinea = respuestaSIM.indexOf('\n', respuestaSIM.indexOf(F("+CMT:")));
        if (saltoDeLinea != -1) {
            ultimoMensaje = respuestaSIM.substring(saltoDeLinea + 1);
            ultimoMensaje.trim();
            hayMensajeNuevo = true;
            actualizarPantalla();
        }
    }
}

}

void procesarModo1(char tecla) {
    if (tecla >= '0' && tecla <= '9') {
        if (numeroCelular.length() < 9) { numeroCelular += tecla; actualizarPantalla(); }
    }
    else if (tecla == '*') {
        numeroCelular = ""; actualizarPantalla();
    }
    else if (tecla == 'D') {
        if (numeroCelular.length() == 9) {
            enviarSMSEmergencia(numeroCelular);
            numeroCelular = ""; actualizarPantalla();
        }
    }
}

void procesarModo2(char tecla) {
    if (pasoModo2 == 0) {
        if (tecla >= '0' && tecla <= '9') {
            if (numeroCelular.length() < 9) { numeroCelular += tecla; actualizarPantalla(); }
        }
        else if (tecla == '*') {
            numeroCelular = ""; actualizarPantalla();
        }
        else if (tecla == 'D' && numeroCelular.length() == 9) {
            pasoModo2 = 1;
            actualizarPantalla();
        }
    }
    else if (pasoModo2 == 1) {
        if (tecla >= '1' && tecla <= '9') {
            int espacio = tecla - '0';
            agenda[espacio] = numeroCelular;
            lcd.clear();
            lcd.setCursor(0,0); lcd.print(F("Guardado en:"));
            lcd.setCursor(0,1); lcd.print(F("Espacio ")); lcd.print(espacio);
            delay(2000);
            numeroCelular = "";
            pasoModo2 = 0;
            modoActual = 3;
            actualizarPantalla();
        }
        else if (tecla == '*') {
            pasoModo2 = 0; actualizarPantalla();
        }
    }
}
}

```

```

void procesarModo3(char tecla) {
    if (tecla >= '1' && tecla <= '9') {
        int espacio = tecla - '0';
        if (agenda[espacio].length() == 9) {
            memoriaTemp = espacio;
            numeroCelular = agenda[espacio];
            actualizarPantalla();
        } else {
            lcd.clear(); lcd.setCursor(0,0); lcd.print(F("Espacio vacio")); delay(1500); actualizarPantalla();
        }
    } else if (tecla == '*') {
        numeroCelular = ""; memoriaTemp = -1; actualizarPantalla();
    } else if (tecla == 'D') {
        if (numeroCelular.length() == 9) {
            enviarSMSEmergencia(numeroCelular);
            numeroCelular = ""; memoriaTemp = -1;
            actualizarPantalla();
        }
    }
}

void actualizarPantalla() {
    lcd.clear();
    if (hayMensajeNuevo) {
        lcd.setCursor(0, 0); lcd.print(F("SMS NUEVO! (A)"));
    } else {
        if (modoActual == 1) { lcd.setCursor(0, 0); lcd.print(F("M1:Escriba Num:")); }
        else if (modoActual == 2) {
            if (pasoModo2 == 0) { lcd.setCursor(0, 0); lcd.print(F("M2:Num a guardar")); }
            else { lcd.setCursor(0, 0); lcd.print(F("M2:Elija slot 1-9")); }
        } else if (modoActual == 3) { lcd.setCursor(0, 0); lcd.print(F("M3:SOS Rapido")); }
    }
    lcd.setCursor(0, 1);
    lcd.print(numeroCelular);
}

void enviarSMSEmergencia(String numero) {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print(F("Enviando SMS..."));
    sim800.println(F("AT+CMGF=1")); delay(300);
    sim800.print(F("AT+CMGS="+51)); sim800.print(numero); sim800.println(F("\n")); delay(1000);
    sim800.print(F("ALERTA S.O.S: Mensaje de emergencia.")); delay(500);
    sim800.write(26);
    for(int i=0; i<6; i++){
        digitalWrite(LED_VERDE, HIGH); delay(400);
        digitalWrite(LED_VERDE, LOW); delay(400);
    }
    digitalWrite(LED_VERDE, HIGH);
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print(F("SMS ENVIADO"));
    delay(2000);
    digitalWrite(LED_VERDE, LOW);
}

void verificarRed() {
    sim800.println(F("AT+CREG?"));
    delay(500);
    if (sim800.available()) {
        String res = sim800.readString();
        if (res.indexOf(F("0,1")) != -1 || res.indexOf(F("0,5")) != -1) {
            redConectada = true;
        } else {
            redConectada = false;
        }
    }
}

```